

Software Engineering

Lecture 3: Agile Software Development

Prof. Nien-Lin Hsueh

Department of Information Engineering and Computer Science
Feng Chia University

Focus Questions

- Why did **Agile development methods** emerge to replace traditional plan-driven approaches?
- What are the four core values of the **Agile Manifesto**?
- What are the key practices of **Extreme Programming (XP)**?
- How are requirements captured using **User Stories** and **Task Cards**?
- What is **Test-Driven Development (TDD)** and why is **Refactoring** critical?
- How does **Scrum** manage iterative development through roles, artifacts, and sprints?

Rapid Software Development

- **Fast-Changing Business Environment:**

- Businesses operate in volatile markets where producing stable, long-term requirements is practically impossible.

- **Evolution Speed:**

- Software must evolve rapidly to reflect changing customer needs.

- **Plan-Driven Limitations:**

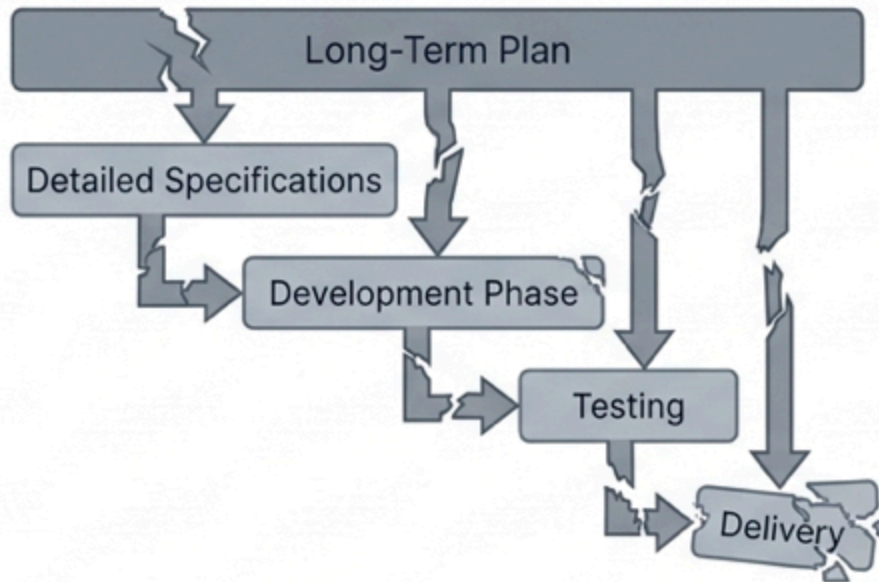
- Heavy plan-driven models create bureaucracy and fail to deliver rapid business value.

- **Rise of Agile:**

- Emerged in the late 1990s to radically reduce delivery time for working software systems.

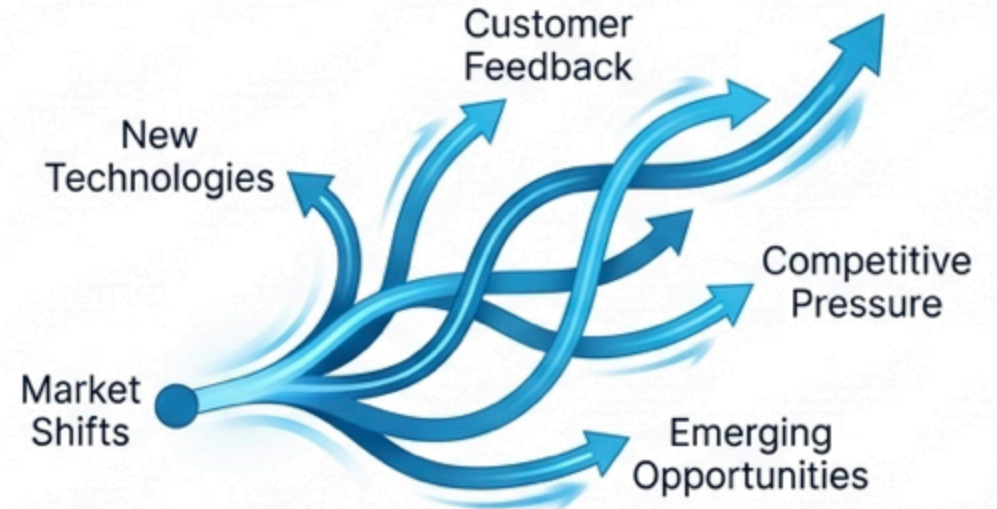
Volatile markets rendered long-term, stable requirements practically impossible.

The 1990s Context



Heavy plan-driven models create bureaucracy.
Failure to deliver rapid business value.

The Fast-Changing Environment



Evolution Speed: Software must evolve rapidly
to reflect changing customer needs.

Emerged in the late 1990s as a radical necessity to slash delivery time for working software systems.

Agile Development Characteristics

- **Interleaved Activities:**
 - Specification, design, and implementation occur concurrently rather than in separate sequential phases.
- **Incremental Versions:**
 - Systems are developed as a series of increments with customer involvement in specification and evaluation.
- **Tool & Automation Driven:**
 - Heavy reliance on automated testing frameworks and continuous integration tools.
- **Minimal Documentation:**
 - Focuses on delivering working code rather than exhaustive design documents.

Agile relies on interleaved activities and automated tools rather than rigid sequential phases.

Traditional (Plan-Driven)

Agile Model

Separate, sequential phases.

Activity Flow

Interleaved activities (specification, design, implementation occur concurrently).

Massive big bang release.

Delivery Method

Incremental versions evaluated by customers.

Exhaustive design documents.

Documentation

Minimal documentation, focus on delivering working code.

Manual hand-offs.

Infrastructure

Heavy reliance on continuous integration tools and automated testing frameworks.

The Agile Manifesto

In 2001, 17 software developers created the **Agile Manifesto**, valuing:

- **Individuals and interactions** over processes and tools
- **Working software** over comprehensive documentation
- **Customer collaboration** over contract negotiation
- **Responding to change** over following a plan

💡 *While there is value in the items on the right, we value the items on the left more.*

”

The 2001 Agile Manifesto shifted focus from rigid contracts to human collaboration.

Individuals and interactions

processes and tools

Working software

comprehensive documentation

Customer collaboration

contract negotiation

Responding to change

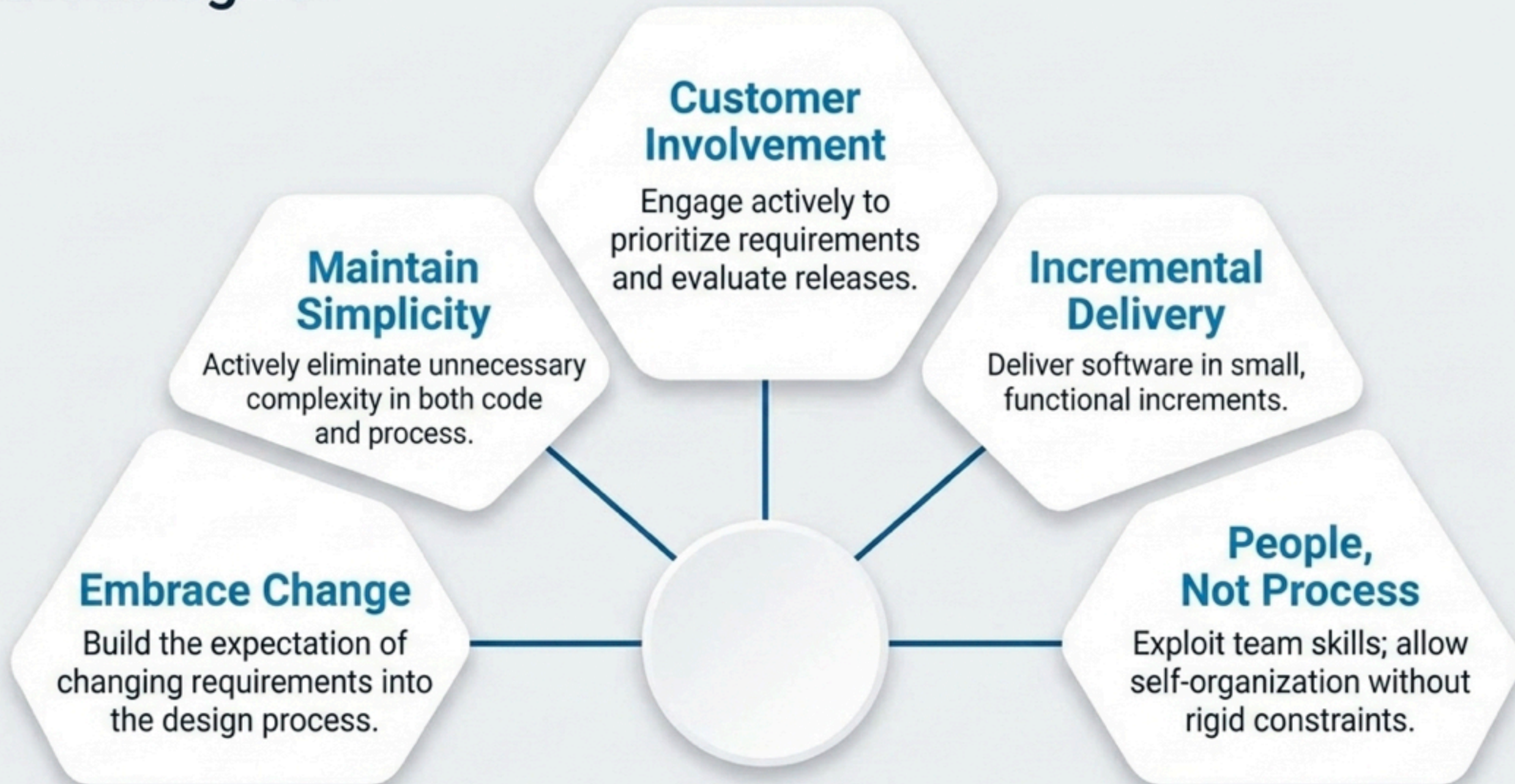
following a plan

While there is value in the items on the right, we value the items on the left more.

5 Core Principles of Agile Methods

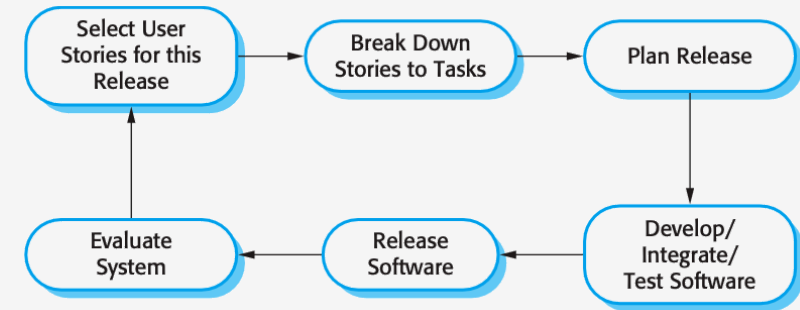
1. **Customer Involvement:** Customers are actively engaged throughout to provide/prioritize requirements and evaluate releases.
2. **Incremental Delivery:** Software is delivered in small, functional increments.
3. **People, Not Process:** Team skills are exploited; developers organize their own work without rigid constraints.
4. **Embrace Change:** Requirements change is expected and built into the design process.
5. **Maintain Simplicity:** Actively eliminate unnecessary complexity in both code and process.

Five core principles form the foundation of agile methodologies.



Extreme Programming (XP)

- **Extreme Approach:** Pushes iterative development practices to extreme levels.
- **Frequent Builds:** New versions built multiple times per day.
- **2-Week Deliveries:** Functional increments delivered to customers every 2 weeks.
- **Automated Safety:** All unit tests must pass before any build is accepted.



Extreme Programming (XP) pushes iterative delivery to its logical extremes.



**Frequent
Builds**

Multiple times per day



**2-Week
Deliveries**

Functional increments
delivered to customers



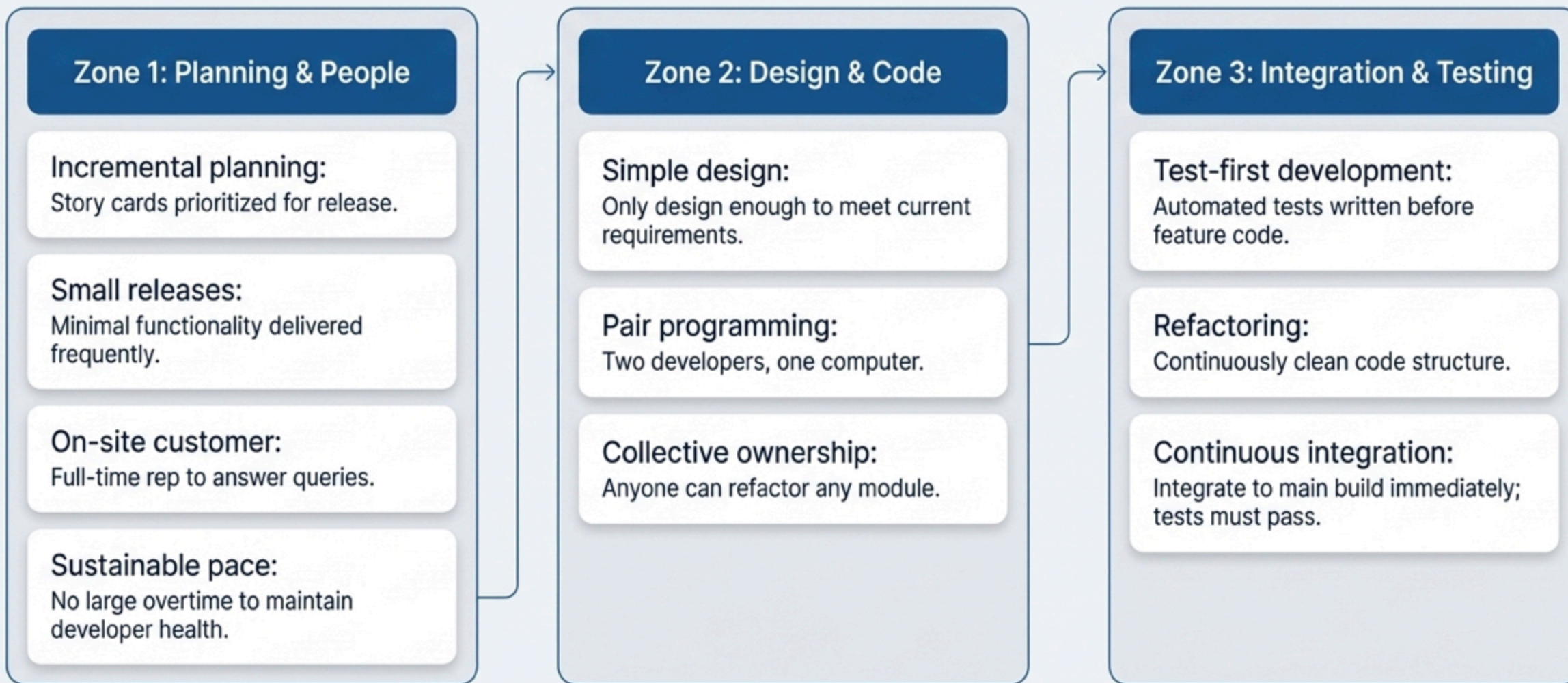
**100%
Passing**

Automated Safety: All unit
tests must pass before any
build is accepted

XP 10 Core Practices

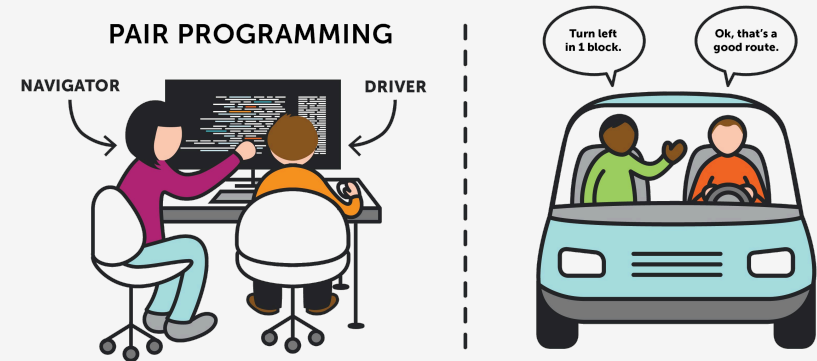
Practice	Key Objective
Incremental planning	Requirements recorded on story cards, prioritized for release.
Small releases	Minimal useful functionality delivered first and frequently updated.
Simple design	Carry out enough design to meet current requirements—no more.
Test-first development	Automated unit tests written <i>before</i> feature code is implemented.
Refactoring	Continuously clean up and improve code structure without changing behavior.
Pair programming	Two developers work at one computer, checking work continuously.
Collective ownership	Developers work across all code; anyone can refactor any module.
Continuous integration	Integrate completed tasks into main build immediately; all tests must pass.
Sustainable pace	No large overtime; maintains code quality and developer health.
On-site customer	Full-time customer representative available to answer requirements queries.

The ten XP practices form a cohesive engine across planning, coding, and integration.

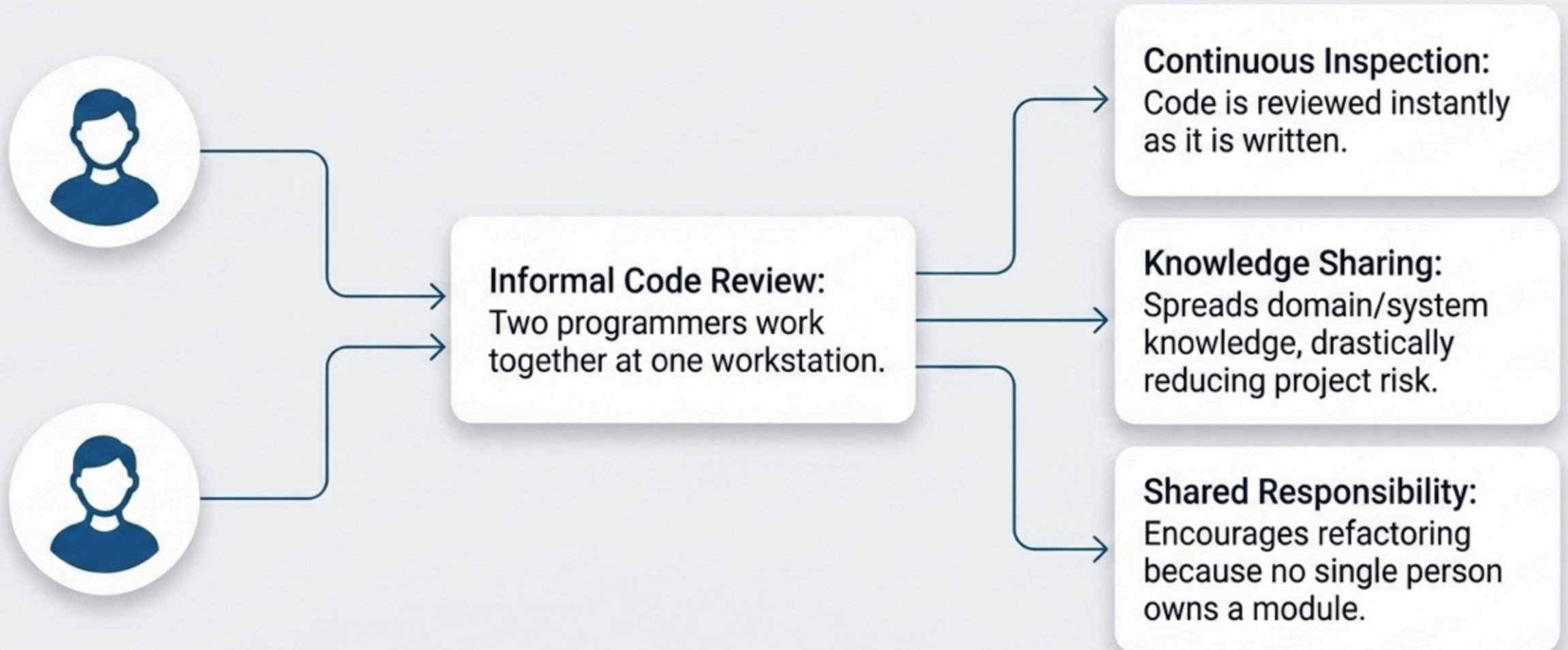


Pair Programming

- **Collaborative Coding:** Two programmers work together at one workstation.
- **Informal Code Review:** Continuous inspection as code is written.
- **Knowledge Sharing:** Spreads domain and system knowledge across the entire team, reducing project risk.
- **Collective Ownership:** Encourages refactoring and shared responsibility.



Collaborative coding diffuses knowledge and ensures collective code ownership.



User Stories & Task Cards

- **Expressing Requirements:**
 - Written from the user's perspective on physical or digital story cards.
- **Task Breakdown:**
 - Developers break each user story down into concrete implementation **tasks**.
- **Estimation & Scheduling:**
 - Tasks are estimated by effort/time, forming the schedule for the next release.
- **Customer Selection:**
 - Customers prioritize user stories to include in the upcoming sprint.

The image shows three overlapping task cards. The top card is titled 'Task 1: Change Dose of Prescribed Drug'. The middle card is titled 'Task 2: Formulary Selection'. The bottom card is titled 'Task 3: Dose Checking' and contains detailed instructions for the dose checking process.

Task 1: Change Dose of Prescribed Drug

Task 2: Formulary Selection

Task 3: Dose Checking

Dose checking is a safety precaution to check that the doctor has not prescribed a dangerously small or large dose.

Using the formulary ID for the generic drug name, look up the formulary and retrieve the recommended maximum and minimum dose.

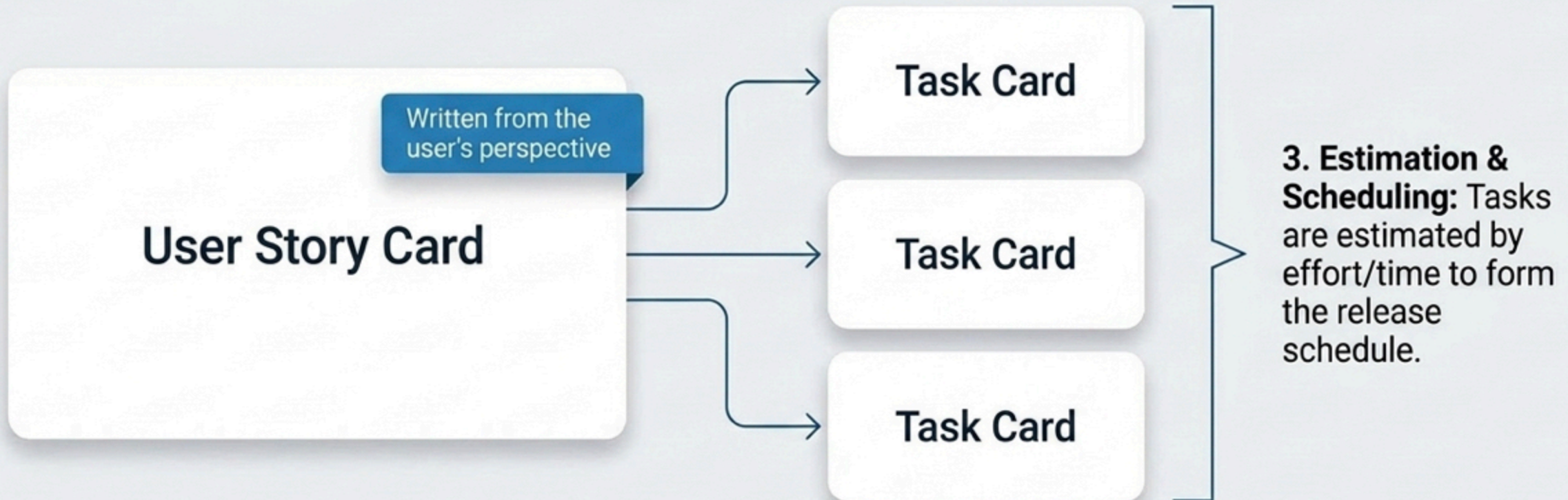
Check the prescribed dose against the minimum and maximum. If outside the range, issue an error message saying that the dose is too high or too low.

If within the range, enable the 'Confirm' button.

Requirements are captured as user stories and translated into actionable development tasks.

1. Customer Selection: Customers prioritize physical/digital story cards for the upcoming sprint.

2. Task Breakdown: Developers break user stories into concrete implementation tasks.



Refactoring

- **Constant Code Improvement:**

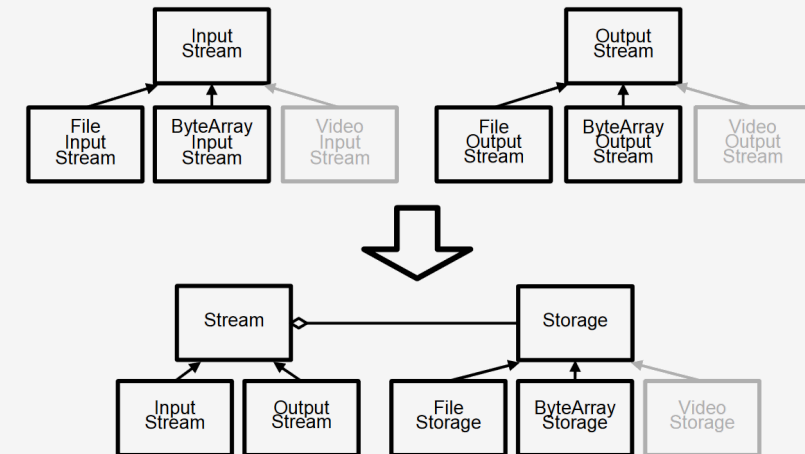
- Restructuring code to make it simpler and more maintainable without altering external functionality.

- **Why Refactor?**

- Prevents code degradation during rapid change.
- Eliminates duplicate code and improves readability.

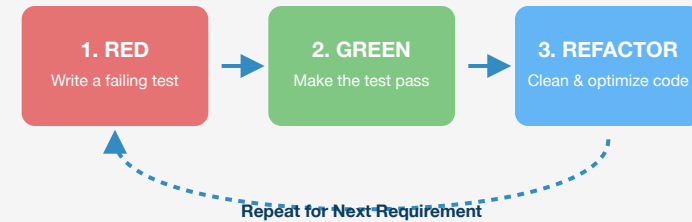
- **Common Refactoring Techniques:**

- *Extract Method*: Splitting complex functions.
- *Rename Variables/Methods*: Improving clarity.

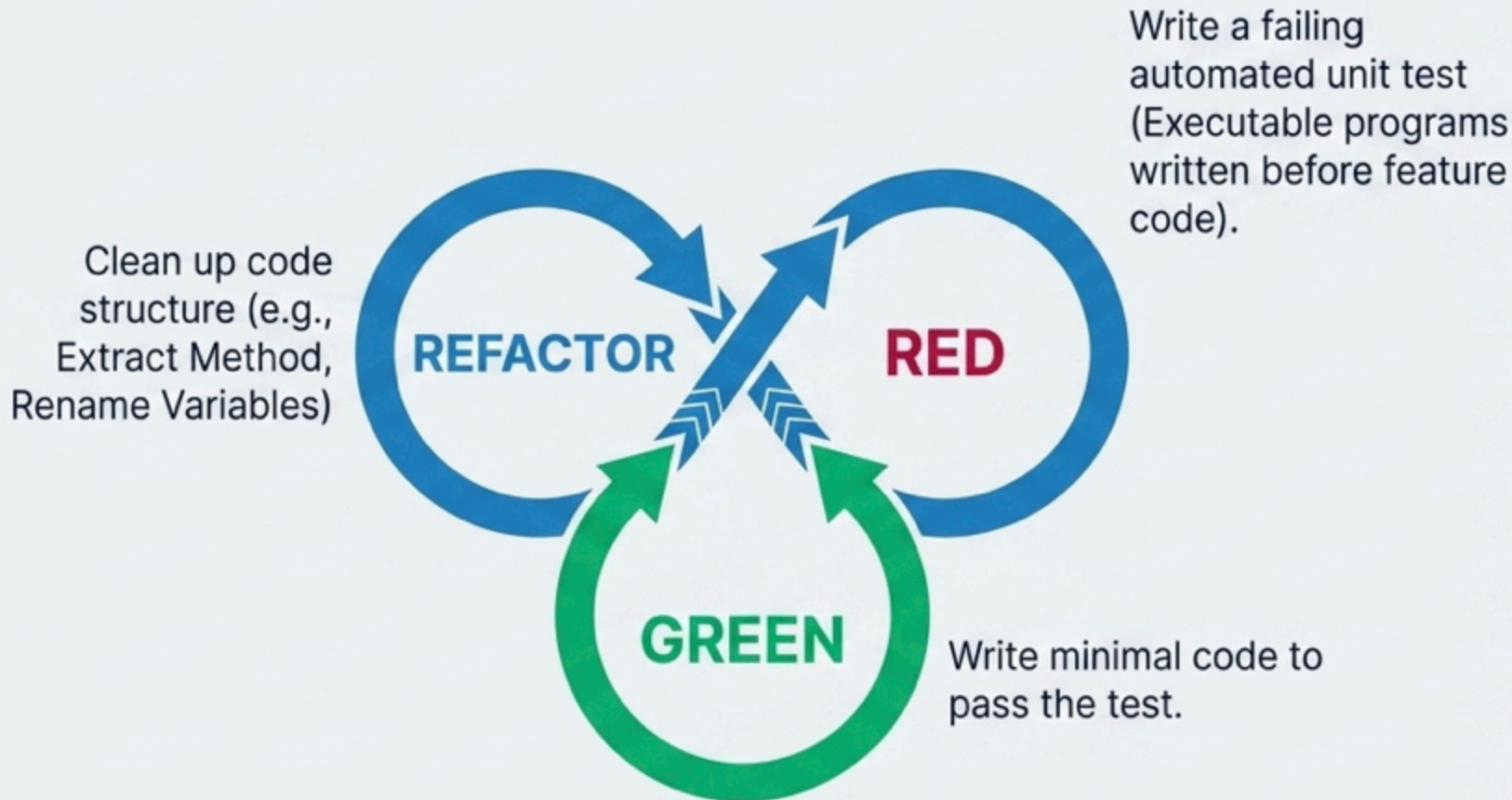


Test-Driven Development (TDD)

- **Test First:** Tests are written as executable programs *before* feature code.
- **The TDD Cycle:**
 1. **RED:** Write a failing automated unit test.
 2. **GREEN:** Write minimal code to pass the test.
 3. **REFACTOR:** Clean up code structure.
- **Regression Testing:** Automated test suites ensure new code doesn't break existing features.



Test-Driven Development acts as a heartbeat, paired with continuous refactoring to prevent degradation.



Regression Testing

Automated test suites ensure new code never breaks existing external functionality.

Eliminates duplicate code and improves readability without altering behavior.

Concept Check Question 1

Which of the following is ONE of the 4 core values stated in the Agile Manifesto?

- **A.** Comprehensive documentation over working software.
- **B.** Following a plan over responding to change.
- **C.** Customer collaboration over contract negotiation.
- **D.** Processes and tools over individuals and interactions.



Concept Check Question 1: Answer

Which of the following is ONE of the 4 core values stated in the Agile Manifesto?

- **Correct Answer: C**
- **Explanation:** The Agile Manifesto values *Customer collaboration over contract negotiation*. The other options invert the manifesto's core values.



Agile Project Management: Scrum

- **Scrum Focus:** An agile framework that focuses on managing iterative development through regular sprints and self-organizing teams.
- **Three Phases of Scrum:**
 1. **Outline Planning:** Setting general objectives and designing overall software architecture.
 2. **Sprint Cycles:** Iterative 2-4 week cycles delivering working software increments.
 3. **Project Closure:** Completing documentation, user manuals, and post-project assessment.

Scrum provides a project management wrapper centered on self-organizing teams.

Outline Planning

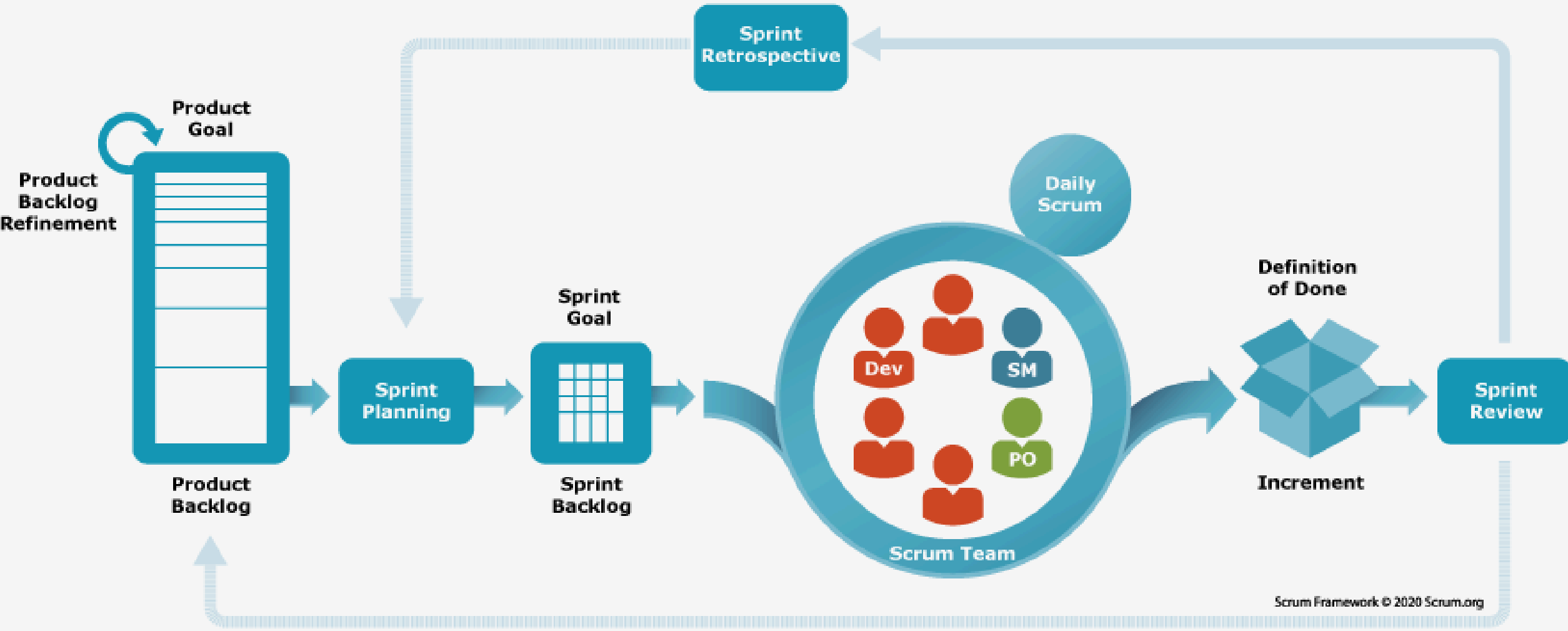
Setting general objectives and designing overall software architecture.

Sprint Cycles

Iterative 2-4 week cycles delivering working software increments.

Project Closure

Completing documentation, user manuals, and post-project assessment.



Key Scrum Roles & Artifacts

- **Scrum Roles:**

- **Product Owner:** Defines product features, prioritizes the Product Backlog, represents business needs.
- **ScrumMaster:** Facilitator who protects the team from external distractions and ensures Scrum process compliance.
- **Development Team:** Self-organizing group (5-9 members) responsible for building increments.

- **Scrum Artifacts:**

- **Product Backlog:** Prioritized list of all features, user stories, and technical requirements.
- **Sprint Backlog:** Sub-set of backlog items selected for the current sprint.
- **Potentially Shippable Increment:** Fully tested, working code delivered at sprint end.

The Scrum ecosystem balances distinct human roles with tangible product artifacts.

The Roles

Product Owner: Defines features, represents business needs, prioritizes backlog

ScrumMaster: Facilitator, protects development team from external distractions, ensures compliance

Development Team: 5-9 members, self-organizing, builds increments

The Artifacts

Product Backlog: Prioritized list of all features/tech requirements

Sprint Backlog: Sub-set of items selected for current sprint

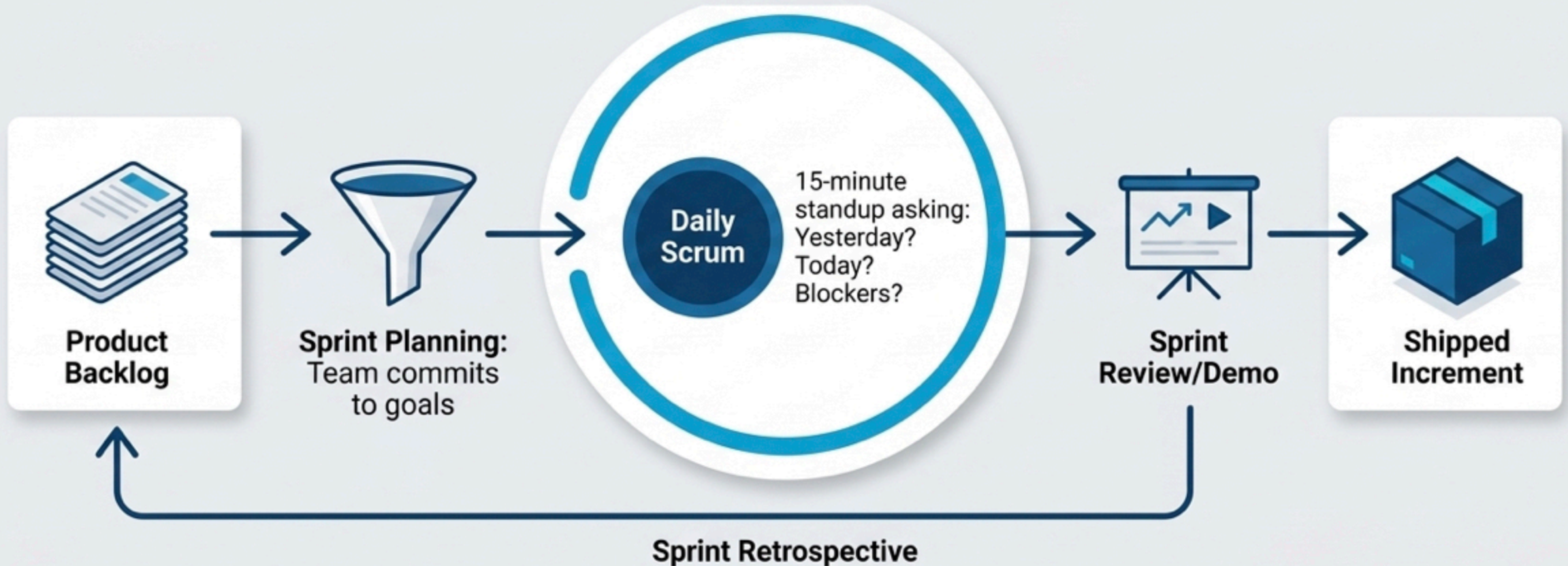
Potentially Shippable Increment: Fully tested, working code delivered at sprint end

Scrum Sprint Ceremonies

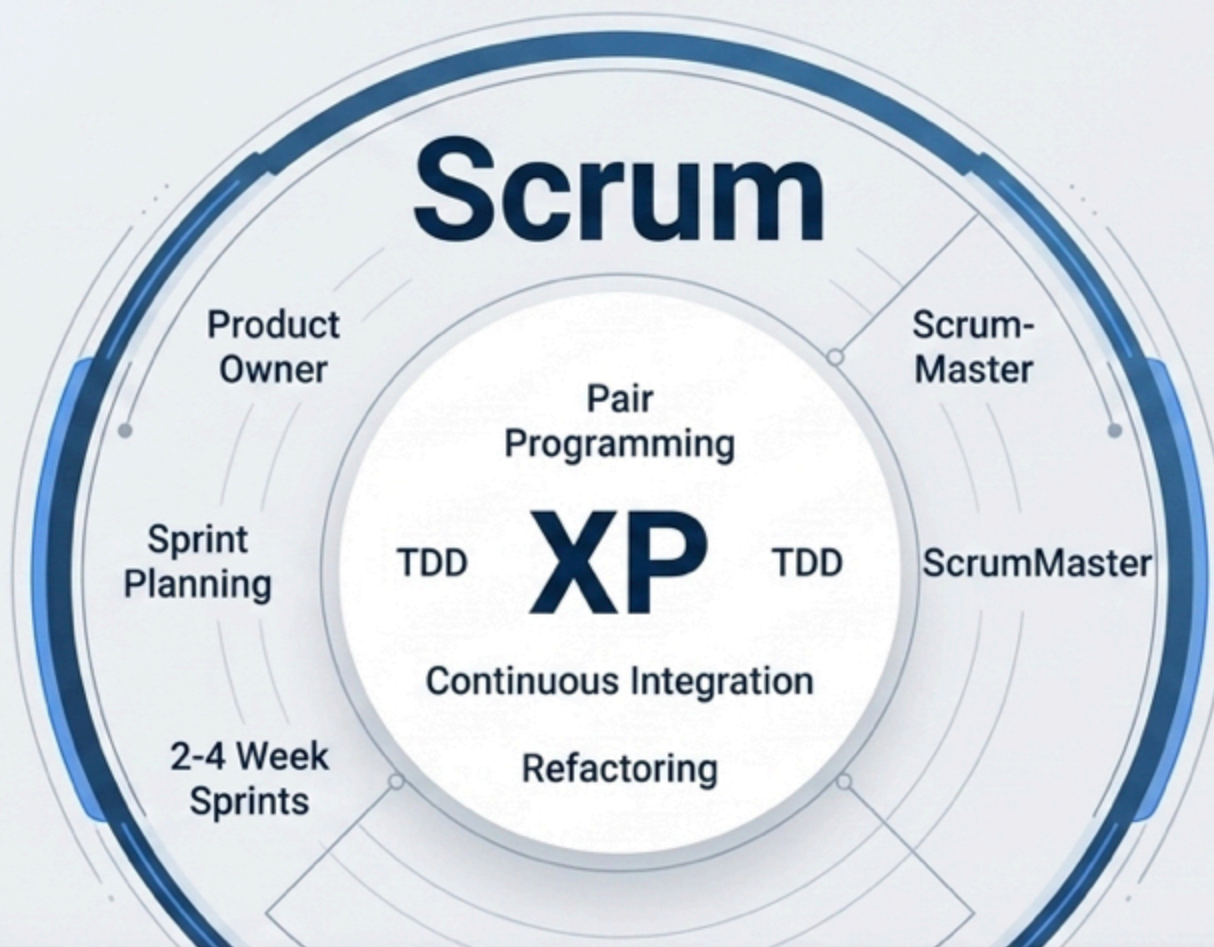
- **Sprint Planning:** Team selects high-priority backlog items and commits to sprint goals.
- **Daily Scrum (Standup):** 15-minute daily meeting where team members review:
 - *What did I do yesterday? What will I do today? What blockers do I have?*
- **Sprint Review (Demo):** Presenting working increment to stakeholders for feedback.
- **Sprint Retrospective:** Recurring meeting to discuss what went well and what can be improved.

Sprints are structured, recurring ceremonies that drive product progression.

The 2-4 Week Cycle Loop



High-performing teams wrap Scrum management around Extreme Programming engineering.



Scrum dictates how to manage the work and self-organize.
Extreme Programming (XP) dictates how to engineer the code with unyielding quality.
They are not mutually exclusive; they are mutually reinforcing.

Concept Check Question 2

In Scrum, who is responsible for protecting the development team from external distractions and facilitating daily meetings?

- **A.** Product Owner
- **B.** ScrumMaster
- **C.** Project Sponsor
- **D.** Lead Architect



Concept Check Question 2: Answer

In Scrum, who is responsible for protecting the development team from external distractions and facilitating daily meetings?

- **Correct Answer: B**
- **Explanation:** The ScrumMaster acts as a facilitator and buffer between external management/distractions and the development team.



Recap: Fill-in-the-blank Quiz

Test your understanding of the core concepts in this chapter:

1. The Agile Manifesto values **working software** over comprehensive ____.
2. In XP, writing automated unit tests *before* writing code is called ____ development.
3. Code ____ involves continuously restructuring existing code without altering its external behavior.
4. In Scrum, the ____ Backlog contains the prioritized list of all features and requirements for the project.



Recap: Answers & Summary

Here are the completed concepts:

1. The Agile Manifesto values **working software** over comprehensive **documentation**.
2. In XP, writing automated unit tests *before* writing code is called **test-first (or TDD)** development.
3. Code **refactoring** involves continuously restructuring existing code without altering its external behavior.
4. In Scrum, the **Product** Backlog contains the prioritized list of all features and requirements for the project.



Validating agile principles and establishing foundations for further exploration.

System Code-Commit Checklist

-  **Values verified:** Customer collaboration prioritized over contract negotiation; working software over comprehensive documentation.
-  **TDD confirmed:** Automated unit tests written prior to feature code (Test-First).
-  **Architecture protected:** Code refactored continuously without altering external behavior.
-  **Workflow secured:** ScrumMaster actively shielding the development team; Product Backlog fully prioritized.

References

- **Sommerville Software Engineering Book**
 - [Official Website](#)
 - [Agile Alliance Manifesto](#)
 - [Scrum Guide Official](#)